

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ae3ec722-12a\agent-acb809bb52b0bbd41.jsonl`
- SHA-256: `f6e478ce9b78e126fcb57809a1bdf1d67d9eea09912c0c85bf735237617f7777`
- Source modified: `2026-07-06T07:58:27+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf\_ae3ec722-12a`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T07:57:10.185Z)

Reviewing GitHub PR #37 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-c-delivery-routing -> main, one commit 100abb2).
It implements ADR 0011 Track C (docs/adr/0011-provider-neutral-domain-schema.md): populates & uses the destinations / routing_rules / delivery_attempts tables that Track B (#35, merged) created empty.
Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/delivery.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff +675/-9.

Key new code:

- db.py: `_migration_004_destinations_and_routing` (backfills destinations from `filter_config_deprecated`, `routing_rules` per source, `delivery_attempts` from `videos_deprecated.forwarded=1`); `repos` `upsert_destination`, `get_destination`, `upsert_delivery_attempt`, `record_delivery` (UPSERT with `attempt_count` increment), `get_delivery_attempt`.
- delivery.py: `forward_to_target/send_link_message/deliver` gain an optional `destination_id` kwarg; when set, `record_delivery` writes a `DeliveryAttempt`; when None (all current live callers) falls back to the `mark_forwarded` no-op.
- models.py: `Destination`, `RoutingRule`, `DeliveryStatus`, `DeliveryAttempt`.

Note: routing is NOT yet wired into the live path (`telethon_client/manager` still call `deliver()` with `destination_id=None`); `destination_id` is currently exercised only by tests. So `record_delivery` issues are LATENT until Track D, but ship now.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run a repro: C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. Build schema via `db._apply_schema(conn)` on `aiosqlite ":memory:"`; or `db.init_db(path)` on a temp file seeded with a legacy v1 schema (`users/channels/videos/filter_config`) to exercise the migration ladder (001->004). NOTE: a legacy 'videos' table MUST include columns `duration_s,width,height,size_bytes,mime_type` or migration 002's column-copy fails (that's a test-harness requirement, not a bug).
- Authoritative local gate ALREADY RUN (don't re-run full suite): `ruff check clean`; `pytest 451` passed @ 95.75% (floor 80%); `ruff format --check flags commands.py` ONLY because this branch predates the merged #36 format fix (`commands.py` is NOT in this PR) ? do NOT report that as a #37 finding.

- Report ONLY defects in THIS PR's diff. Distinguish LIVE-NOW (migration runs on upgrade) from LATENT
(delivery recording, unused until Track D). No praise. Concrete failure scenario per finding. Rank severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour on upgrade), high (real bug, narrow trigger),
medium (latent-but-real logic bug), low (hygiene/edge). Be honest; downgrade unrealistic triggers.

ADVERSARIALLY VERIFY this single finding ? try to REFUTE it. Read the exact cited code, trace real control flow, and run a repro with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour (note live vs latent). REFUTED if the code handles it, the trigger can't occur, or it's pre-existing/not in this diff. PLAUSIBLE if you can't fully settle it. Give corrected severity.

FINDING (dim migration_004):
title: Malformed/non-object config_json aborts the entire migration ladder on upgrade
severity(claimed): medium | live
where: src/tg_video_filter/db.py:694
summary: Migration 004 parses filter_config_deprecated.config_json with json_extract(); if any row's config_json is not valid JSON, json_extract raises sqlite3.OperationalError and the whole init_db upgrade fails, so the app cannot start.
failure_scenario: A legacy DB where any filter_config row's config_json is not strictly-valid JSON (hand-edited, truncated on a prior crash, or written by an older/foreign tool). On the first upgrade, _migration_004 executes step 1's INSERT ... SELECT COALESCE(json_extract(config_json, '\$.delivery_target'), 'me') ... FROM filter_config_deprecated. json_extract aborts with 'malformed JSON', _apply_migrations propagates the exception, the ladder rolls back, and every subsequent startup re-hits the same crash. Reproduced: seeding a filter_config row with config_json='not-json-at-all' makes init_db raise sqlite3.OperationalError: malformed JSON (scenario G).
evidence: Repro (scratchpad/repro5.py, scenario G) seeds a legacy DB with config_json='not-json-at-all' and init_db() raises sqlite3.OperationalError: malformed JSON at db.py:690 (the step-1 INSERT). Migration 003 previously only copied config_json verbatim into policies (db.py:624) and never validated it, so 004 is the first consumer to parse it. Reachability is low because the live save_filter_config always writes cfg.model_dump_json() (db.py:1684), i.e. valid JSON ? hence medium, not blocker.
recommendation: Guard the parse against malformed JSON, e.g. wrap the extract in CASE WHEN json_valid(config_json) THEN json_extract(config_json, '\$.delivery_target') ELSE NULL END (also in the routing_rules subquery at db.py:718), so a corrupt row degrades to the 'me' default instead of aborting the whole upgrade.

2. assistant (2026-07-06T07:57:13.564Z)

I'll verify this finding by reading the exact cited code and running a repro.

3. user (2026-07-06T07:57:14.720Z)

```
660         # 6. Rename (don't drop) the legacy tables ? kept for one release as a
661         #     rollback safety net (ADR 0010 pattern).
662         await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
663         await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
664         await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
665
666
667     @_migration(4)
668     async def _migration_004_destinations_and_routing(conn: aiosqlite.Connection) -> None:
669         """Backfill default destinations and delivery state (ADR 0011, Track C).
670
671         Creates a default destination row for each account that has a
672         ``filter_config_deprecated.delivery_target`` value, then creates a default
673         routing rule (source ? default policy ? destination) for each source.
674         Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
```

```

675     table (one ``delivered`` attempt per previously-forwarded video to the
676     account's default destination).
677
678     Idempotent: skips accounts whose default destination already exists.
679     """
680     # Gate: if destinations already have data, this migration has run.
681     async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682         if (await cur.fetchone())[0] > 0:
683             return
684
685     if not await _table_exists(conn, "filter_config_deprecated"):
686         return
687
688     # 1. Backfill default destinations from filter_config_deprecated.
689     #     Parse the JSON delivery_target, defaulting to "me".
690     await conn.execute(
691         """
692         INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
is_output, created_at)
693         SELECT user_id, 'telegram',
694             COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),
695             CASE
696                 WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') =
'me'
697                 THEN 'saved'
698                 ELSE 'channel'
699             END,
700             0,
701             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
702         FROM filter_config_deprecated
703         """
704     )
705
706     # 2. Create a default routing rule per source (source ? default policy ? default
destination).
707     await conn.execute(
708         """
709         INSERT OR IGNORE INTO routing_rules
710             (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
711         SELECT s.account_id, s.id, p.id, d.id, 1, 0,
712             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
713         FROM sources s
714         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
715         JOIN destinations d ON d.account_id = s.account_id
716         WHERE d.target = (
717             SELECT COALESCE(
718                 json_extract(fcd.config_json, '$.delivery_target'), 'me'
719             )
720         FROM filter_config_deprecated fcd
721         WHERE fcd.user_id = s.account_id
722         LIMIT 1
723         )
724         """
725     )
726
727     # 3. Backfill delivery_attempts from previously-forwarded videos.
728     await conn.execute(
729         """
730         INSERT OR IGNORE INTO delivery_attempts
731             (content_item_id, destination_id, route_id, status, attempt_count,
732             created_at, delivered_at)
733         SELECT ci.id, d.id, rr.id, 'delivered', 1,
734             vd.seen_at, vd.seen_at
735         FROM videos_deprecated vd

```

```

736         JOIN content_items ci
737         ON ci.account_id = vd.user_id
738         AND ci.provider = 'telegram'
739         AND ci.provider_message_id = CAST(vd.message_id AS TEXT)

```

4. assistant (2026-07-06T07:57:17.585Z)

Let me see the rest of the migration and check the schema for whether config_json can be malformed.

5. user (2026-07-06T07:57:18.625Z)

```

740         JOIN sources sc
741         ON sc.id = ci.source_id
742         AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
743     JOIN destinations d ON d.account_id = vd.user_id
744     JOIN routing_rules rr
745     ON rr.account_id = vd.user_id
746     AND rr.source_id = sc.id
747     AND rr.destination_id = d.id
748     WHERE vd.forwarded = 1
749     """
750 )
751
752
753 async def _table_exists(conn: aiomysqlite.Connection, table: str) -> bool:
754     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
755     async with conn.execute(
756         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
757     ) as cur:
758         return await cur.fetchone() is not None
759
760
761 # --- helpers -----
762
763
764 def _now() -> datetime:
765     """Timezone-aware UTC 'now', used for policy timestamps."""
766     return datetime.now(UTC)
767
768
769 def _safe_int_cast(value: object, field_name: str) -> int:

```

6. assistant (2026-07-06T07:57:21.930Z)

Now let me run the repro to verify the behavior of json_extract on malformed JSON.

7. user (2026-07-06T07:57:25.657Z)

ERROR: OperationalError malformed JSON

8. assistant (2026-07-06T07:57:30.637Z)

Confirmed json_extract raises on malformed JSON. Now let me run the full migration ladder repro to confirm init_db actually crashes. Let me first check the schema seeding requirements and _apply_migrations behavior.

9. user (2026-07-06T07:57:31.824Z)

```

189:async def init_db(db_path: str) -> None:
199:async def _apply_schema(conn: aiomysqlite.Connection) -> None:
315:async def _apply_migrations(conn: aiomysqlite.Connection) -> None:

```

10. user (2026-07-06T07:57:35.095Z)

```

189     async def init_db(db_path: str) -> None:
190         """Open the database at ``db_path`` and create/migrate the schema.
191
192         Safe to call on every startup. Applies the base schema (`IF NOT EXISTS`)
193         then runs any pending migrations via the ``user_version`` ladder.
194         """
195         async with aiosqlite.connect(db_path) as conn:
196             await _apply_schema(conn)
197
198
199     async def _apply_schema(conn: aiosqlite.Connection) -> None:
200         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
201         # and a busy timeout so concurrent access waits instead of failing immediately.
202         await conn.execute("PRAGMA journal_mode = WAL")
203         await conn.execute("PRAGMA foreign_keys = ON")
204         await conn.execute("PRAGMA busy_timeout = 5000")
205         await conn.executescript(_SCHEMA)
206         # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
207         # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
208         # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
209         # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
210         # any *empty* recreations of those legacy tables from a subsequent
211         # ``init_db`` are dropped here so they don't shadow the renamed copies.
212         if await _table_exists(conn, "channels_deprecated"):
213             await conn.execute("DROP TABLE IF EXISTS channels")
214             await conn.execute("DROP TABLE IF EXISTS videos")
215             await conn.execute("DROP TABLE IF EXISTS filter_config")
216         await _apply_migrations(conn)
217         await conn.commit()
218

```

11. assistant (2026-07-06T07:57:38.786Z)

Now let me look at how `filter_config` is structured in the legacy schema and how migration 003 handles `config_json`, to check the reachability claim. Let me look at the existing test setup for the migration ladder.

12. user (2026-07-06T07:57:40.058Z)

```

52:     # the legacy channels/videos/filter_config are absent on fresh DBs
449:async def test_load_filter_config_defaults_when_absent(conn: aiosqlite.Connection) -> None:
451:     cfg = await db.load_filter_config(conn, USER_ID)
455:async def test_save_and_load_filter_config_round_trip(conn: aiosqlite.Connection) -> None:
460:     await db.save_filter_config(conn, USER_ID, custom)
461:     loaded = await db.load_filter_config(conn, USER_ID)
465:async def test_save_filter_config_overwrites(conn: aiosqlite.Connection) -> None:
467:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=100))
468:     await db.save_filter_config(conn, USER_ID, FilterConfig(max_duration_s=200))
469:     loaded = await db.load_filter_config(conn, USER_ID)
473:async def test_filter_config_with_null_bounds_round_trips(conn: aiosqlite.Connection) ->
None:
479:     await db.save_filter_config(conn, USER_ID, cfg)
480:     loaded = await db.load_filter_config(conn, USER_ID)
493:     await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_mode=mode))
494:     loaded = await db.load_filter_config(conn, USER_ID)
498:async def test_filter_config_rejects_inverted_duration_bounds() -> None:
508:def test_filter_config_accepts_equal_min_max_duration() -> None:
523:     # into the default policy (ADR 0011: filter_config is now policies).
526:     "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
531:     loaded = await db.load_filter_config(conn, USER_ID)
537:async def test_delivery_target_round_trips(conn: aiosqlite.Connection) -> None:
538:     """The delivery_target field survives the JSON round trip for all forms."""
541:     await db.save_filter_config(conn, USER_ID, FilterConfig(delivery_target=target))
542:     loaded = await db.load_filter_config(conn, USER_ID)
543:     assert loaded.delivery_target == target

```

```

546:async def test_old_config_without_delivery_target_defaults_to_me(
549:     """A pre-v1.2 config JSON (no delivery_target key) must default to 'me'."""
556:     "INSERT OR REPLACE INTO policies (account_id, name, config_json, is_default, "
561:     loaded = await db.load_filter_config(conn, USER_ID)
562:     assert loaded.delivery_target == "me"
565:def test_filter_config_coerces_int_delivery_target() -> None:
569:     cfg = FilterConfig(delivery_target=-1001234567890) # type: ignore[arg-type]
570:     assert cfg.delivery_target == "-1001234567890"
573:def test_filter_config_rejects_empty_delivery_target() -> None:
574:     """An empty delivery_target is invalid (min_length=1)."""
580:     FilterConfig(delivery_target="")
665:     CREATE TABLE filter_config (
666:         user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
684:         await conn.execute("INSERT INTO filter_config (user_id, config_json) VALUES (111,
685:         '{}')")
718:         assert await db._table_exists(conn, "filter_config_deprecated")

```

[Showing results with pagination = limit: 40]

13. assistant (2026-07-06T07:57:44.694Z)

Let me look at the existing test that seeds the legacy DB (around line 660-720) to build a faithful repro.

14. user (2026-07-06T07:57:45.541Z)

```

630         async with conn.execute("PRAGMA user_version") as cur:
631             version = (await cur.fetchone())[0]
632             assert version == len(db._MIGRATIONS)
633             assert version >= 1
634
635
636     async def test_migration_adds_document_id_to_legacy_db(tmp_path: object) -> None:
637         """A v1 DB (no document_id, user_version=0) must be migrated on open."""
638         db_path = str(tmp_path) + "/legacy.db" # type: ignore[operator]
639
640         # Build a legacy v1 schema WITH the real FK definitions (REFERENCES users),
641         # no document_id column, user_version = 0. Using the real FK constraints
642         # is what lets this test catch the PR #33 regression where SQLite rewrites
643         # FK references to follow a rename (users ? users_deprecated) instead of
644         # retargeting them to accounts.
645         legacy_schema = """
646         CREATE TABLE users (
647             id INTEGER PRIMARY KEY, phone TEXT,
648             enabled INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
649         );
650         CREATE TABLE channels (
651             id INTEGER PRIMARY KEY AUTOINCREMENT,
652             user_id INTEGER NOT NULL REFERENCES users(id),
653             channel_id INTEGER NOT NULL, title TEXT, seen_first_at TEXT NOT NULL,
654             UNIQUE (user_id, channel_id)
655         );
656         CREATE TABLE videos (
657             id INTEGER PRIMARY KEY AUTOINCREMENT,
658             user_id INTEGER NOT NULL REFERENCES users(id),
659             channel_id INTEGER NOT NULL, message_id INTEGER NOT NULL,
660             duration_s INTEGER, width INTEGER, height INTEGER, size_bytes INTEGER,
661             mime_type TEXT, matched INTEGER NOT NULL DEFAULT 0,
662             forwarded INTEGER NOT NULL DEFAULT 0, seen_at TEXT NOT NULL,
663             UNIQUE (user_id, channel_id, message_id)
664         );
665         CREATE TABLE filter_config (
666             user_id INTEGER PRIMARY KEY REFERENCES users(id), config_json TEXT NOT NULL
667         );
668         """

```

```

669         async with aiosqlite.connect(db_path) as conn:
670             await conn.executescript(legacy_schema)
671             # Seed rows so we can confirm data survives the migration.
672             await conn.execute(
673                 "INSERT INTO users (id, phone, enabled, created_at) VALUES (111, '+1', 1,
674                 '2024-01-01T00:00:00Z')"
675             )
676             await conn.execute(
677                 "INSERT INTO channels (user_id, channel_id, title, seen_first_at) "
678                 "VALUES (111, -1001, 'pre-migration', '2024-01-01T00:00:00Z')"
679             )
680             await conn.execute(
681                 "INSERT INTO videos (user_id, channel_id, message_id, duration_s, width,
682                 height, "
683                 "size_bytes, mime_type, matched, forwarded, seen_at) "
684                 "VALUES (111, -1001, 1, 60, 1920, 1080, 1000000, 'video/mp4', 1, 1,
685                 '2024-01-01T00:00:00Z')"
686             )
687             await conn.execute("INSERT INTO filter_config (user_id, config_json) VALUES
688             (111, '{}')")
689             await conn.commit()
690             # user_version is 0 on a freshly created DB.
691
692     # Now open via init_db ? migrations 001, 002, and 003 must run.
693     await db.init_db(db_path)
694
695     async with aiosqlite.connect(db_path) as conn:
696         await conn.execute("PRAGMA foreign_keys=ON")
697         # Column added (migration 001) ? on the legacy videos table, now renamed.
698         assert await db._column_exists(conn, "videos_deprecated", "document_id")
699         # Identity split (migration 002): users renamed, accounts present.
700         assert await db._table_exists(conn, "accounts")
701         assert await db._table_exists(conn, "telegram_sessions")
702         assert await db._table_exists(conn, "users_deprecated")
703         assert not await db._table_exists(conn, "users")
704         # accounts.id copied from users.id (integer-preserving).
705         async with conn.execute("SELECT id, telegram_id, provider FROM accounts") as
706         cur:
707             rows = await cur.fetchall()
708             assert rows == [(111, 111, "telegram")]
709             # telegram_sessions row derived from the legacy user row.
710             async with conn.execute(
711                 "SELECT account_id, phone, session_path, enabled FROM telegram_sessions"
712             ) as cur:
713                 srows = await cur.fetchall()
714                 assert srows == [(111, "+1", "user_111.session", 1)]
715
716         # Domain schema (migration 003): legacy tables renamed to *_deprecated,
717         # provider-neutral tables backfilled from them.
718         assert await db._table_exists(conn, "sources")
719         assert await db._table_exists(conn, "content_items")
720         assert await db._table_exists(conn, "policies")
721         assert await db._table_exists(conn, "channels_deprecated")
722         assert await db._table_exists(conn, "videos_deprecated")
723         assert await db._table_exists(conn, "filter_config_deprecated")
724         assert not await db._table_exists(conn, "channels")
725         assert not await db._table_exists(conn, "videos")
726         assert not await db._table_exists(conn, "filter_config")
727
728         # FK contract (PR #33 review): the rebuilt legacy tables (now
729         # *_deprecated) reference accounts(id), NOT users_deprecated(id).
730         for tbl in ("channels_deprecated", "videos_deprecated",
731 "filter_config_deprecated"):
732             async with conn.execute(f"PRAGMA foreign_key_list({tbl})") as cur:
733                 fk_rows = await cur.fetchall()

```

```
728             referenced_tables = {r[2] for r in fk_rows}
729             assert "accounts" in referenced_tables, f"{tbl} FK does not reference
accounts"
```

15. assistant (2026-07-06T07:57:48.359Z)

Now I have a faithful template. Let me build the full repro with malformed config_json to confirm init_db crashes end-to-end.

16. user (2026-07-06T07:58:00.113Z)

```
cfg='not-json-at-all'    -> OperationalError: malformed JSON
cfg='{'                  -> init_db OK
cfg='{"delivery_target": "me"}' -> init_db OK
```

17. assistant (2026-07-06T07:58:03.466Z)

Confirmed end-to-end. `init\_db` raises `sqlite3.OperationalError: malformed JSON` when a legacy `filter\_config` row holds non-JSON, while valid JSON rows migrate fine. Now let me verify the reachability claim: does the live save path always write valid JSON, and is migration 003 truly the first non-validating consumer?

18. user (2026-07-06T07:58:04.283Z)

```
610             AND height IS NOT NULL AND width IS NOT NULL
611             AND dedup_key LIKE 'tg:msg:%'
612             GROUP BY account_id, size_bytes, duration_s, height, width
613         )
614     """
615 )
616
617 # 3. Backfill policies from filter_config: one 'default' policy per account
618 #   (Option 2A). is_default=1 so the legacy load_filter_config shim reads
619 #   this row. UNIQUE(account_id, name) dedups idempotent re-runs.
620 await conn.execute(
621     """
622     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
623                                     created_at, updated_at)
624     SELECT user_id, 'default', config_json, 1,
625            strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
626            strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
627     FROM filter_config
628     """
629 )
630
631 # 4. Create the policies default-partial index (sig after backfill so the
632 #   UNIQUE constraint only applies once rows are populated). On fresh DBs
633 #   _SCHEMA creates it; on migrated DBs this is the first time.
634 await conn.execute(
635     "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
636     "ON policies(account_id) WHERE is_default = 1"
637 )
638
639 # 5. Assert no silent video loss: on a legacy DB, every videos row MUST
```

19. assistant (2026-07-06T07:58:09.335Z)

Confirmed: migration 003 (line 624) copies `config\_json` verbatim into `policies` without parsing it ? so it never validates JSON. Migration 004 is genuinely the first consumer to parse it via `json\_extract`. Let me verify the live save path writes valid JSON.

20. user (2026-07-06T07:58:10.213Z)

```
1675     async def save_filter_config(conn: aiosqlite.Connection, user_id: int, cfg:
```



```

FilterConfig) -> None:
1676     """Deprecated shim: upsert the account's default :class:`Policy`."""
1677     existing = await get_default_policy(conn, user_id)
1678     await upsert_policy(
1679         conn,
1680         Policy(
1681             id=existing.id if existing is not None else None,
1682             account_id=user_id,
1683             name="default",
1684             config_json=cfg.model_dump_json(),
1685             is_default=True,
1686             created_at=(existing.created_at if existing is not None else _now()),
1687             updated_at=_now(),
1688         ),
1689     )
1690
1691
1692     # --- bulk helpers (used by the manager on startup) -----
1693
1694

```

21. assistant (2026-07-06T07:58:18.565Z)

Confirmed. The live save path always uses `cfg.model\_dump\_json()` (valid JSON), so in normal operation no malformed row exists. The trigger requires a hand-edited/corrupted/foreign-written row.

All claims in the finding are verified:

- `json_extract` raises `OperationalError: malformed JSON` on invalid JSON (repro confirmed).
- ``init_db`` propagates it and aborts the whole ladder end-to-end (full-ladder repro confirmed, at `db.py:690` step-1 INSERT).
- Migration 003 (`db.py:624`) copies `config_json` verbatim, never parsing it, so 004 is the first parser.
- The `routing_rules` subquery at `db.py:718` has the same unguarded ``json_extract``.
- Reachability is low: the live save path always writes valid JSON (`db.py:1684`).

This is LIVE-NOW (migration runs on any upgrade), but with a narrow trigger (only externally-corrupted/hand-edited `config_json`). Severity medium is accurate ? it is a real upgrade-crash that bricks startup for the affected DB, but it cannot arise from data this app itself wrote. Not a blocker because the app never produces malformed `config_json`; not low because when triggered it fully prevents startup with no auto-recovery.

22. user (2026-07-06T07:58:27.496Z)

Structured output provided successfully